

CC58 - Tópicos en Ciencia de la Computación

Trabajo 1: Constraint Programming

Integrantes por grupo: 2 - 3 alumnos.

Entrega del proyecto: Semana 7.

1. Objetivo del Proyecto

El objetivo de este trabajo es que los estudiantes integren técnicas de Inteligencia Artificial (específicamente Visión Computacional) con la Programación con Restricciones (Constraint Programming). Los equipos deberán desarrollar un sistema o pipeline completo ("End-to-End") capaz de "leer una imagen de un acertijo lógico matemático, comprender sus reglas y estado inicial, y resolverlo de manera óptima utilizando un solver de CP.

2. Descripción del Problema

Cada grupo deberá seleccionar **UNO** de los siguientes acertijos lógicos para su proyecto:

1. **Murdoku:** (<https://nl.wikipedia.org/wiki/Murdoku>) es un rompecabezas lógico que combina la estructura del sudoku con el elemento de juego de los misterios de asesinato clásicos como Cluedo. Detectar la cuadrícula de $n \times n$, reconocer los items iniciales impresos en las celdas y resolver el tablero respetando las reglas.
2. **KenKen:** (<https://en.wikipedia.org/wiki/KenKen>) Detectar la topología del tablero (tamaño de la grilla), identificar los bordes gruesos que forman las "jaulas" (*cages*), leer el número objetivo y la operación matemática (+, −, ×, ÷) de cada jaula, y resolverlo.
3. **Kakuro:** (<https://en.wikipedia.org/wiki/Kakuro>) Detectar la grilla, diferenciar entre celdas blancas (vacías) y negras (pistas), usar OCR para leer las sumas objetivo (verticales y horizontales) en las celdas diagonales, y resolver el puzzle sin repetir números en la misma suma.
4. **Futoshiki:** (<https://en.wikipedia.org/wiki/Futoshiki>) Detectar la grilla, los números iniciales dados y, lo más importante, identificar correctamente los signos de desigualdad (< y >) entre celdas adyacentes para formular las restricciones relacionales.

3. Fases del Desarrollo

3.1. Fase 1: Visión Computacional e IA (Extracción de Datos)

El sistema debe recibir una imagen (.jpg, .png) del acertijo elegido.

En esta etapa se espera que el equipo utilice herramientas de Visión Computacional de su elección (ej. OpenCV) y Machine Learning/Deep Learning (ej. Tesseract OCR, YOLO, o una red neuronal convolucional propia entrenada con PyTorch/TensorFlow) para:

- Aplicar preprocesamiento a la imagen (escala de grises, binarización, corrección de perspectiva).
- Segmentar la cuadrícula y las celdas individuales.
- Clasificar el contenido de cada celda (dígitos, operadores matemáticos, signos de desigualdad o formas de las jaulas).
- Generar una estructura de datos (ej. JSON, matriz en Python) que represente el estado inicial del problema.

Se puede utilizar cualquier modelo preentrenado o de la literatura, debidamente referenciado.

3.2. Fase 2: Constraint Programming (Modelado y Resolución)

Utilizando la estructura de datos extraída en la Fase 1, el equipo debe construir un modelo de Constraint Programming formal.

- **Herramientas sugeridas:** OR-Tools (Python), MiniZinc, Choco Solver, o Gecode.
- **Requisitos del modelo:**
 - Definición clara de **variables** y **dominios**.
 - Uso de restricciones binarias y, fundamentalmente, **restricciones globales** (como `alldifferent`, `sum`, etc.).
 - El modelo debe ser parametrizado (debe funcionar para cualquier instancia del puzzle extraída, no estar "hardcodeado").

3.3. Fase 3: Integración y Visualización

- El sistema debe integrar la salida de la Fase 1 como entrada de la Fase 2 de manera automática.
- Una vez el solver encuentre la solución, esta debe proyectarse de manera visual y comprensible (ya sea superponiendo los números generados sobre la imagen original, o dibujando una interfaz gráfica/grilla limpia con el resultado final).

4. Entregables

1. **Código Fuente:** Repositorio en GitHub con el código completo, ordenado y comentado. Debe incluir un archivo `README.md` con las instrucciones exactas de instalación (ej. `requirements.txt`) y ejecución.
2. **Dataset de Prueba:** Un conjunto de al menos 10 imágenes del puzzle elegido en distintas condiciones (diferente iluminación, ángulos ligeros, versiones digitales e impresas) para probar la robustez de la Visión Computacional.
3. **Informe Técnico (PDF formato IEEE o similar):**
 - Explicación del pipeline de Visión Computacional (algoritmos usados y métricas de error).
 - Modelo Matemático formal de CP (Variables, Dominios, Restricciones).
 - Análisis de complejidad y tiempos de respuesta del Solver.
4. **Presentación / Video Demostrativo:** Un video de máximo 5 minutos (o presentación en vivo) donde los 3 miembros del grupo expliquen el código y demuestren el sistema funcionando desde la carga de la foto hasta la visualización de la solución.

5. Rúbrica de Evaluación

Criterio	Descripción	Puntos
Visión Computacional e IA		
	Precisión en la detección de la cuadrícula, números y símbolos lógicos.	3
	Bajo diferentes condiciones de imagen.	1
Modelado CP		
	Correcta formulación del problema matemático.	3
	Uso eficiente de restricciones globales.	3
	Uso eficiente de restricciones reificadas (si fuera necesario, sino explicar porque no).	1
Integración de Software		
	El puente entre la IA y el modelo CP funciona sin intervención manual.	1
	La solución se muestra visualmente de forma clara.	1
Calidad y Documentación		
	Código limpio modularizado y uso de buenas prácticas.	2
	Informe técnico claro y sustentado en Latex con formato artículo.	5